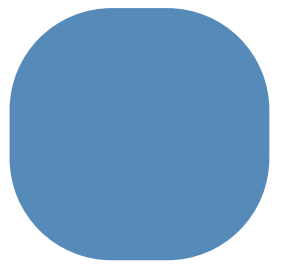


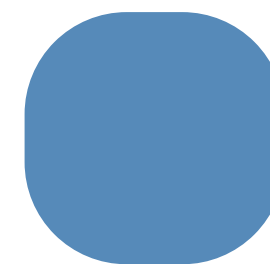
INTRO A WEBSOCKETS



¿Qué son?

Los WebSockets permiten que el **servidor y los clientes mantengan una conexión** abierta y se comuniquen **en tiempo real**, sin tener que hacer peticiones HTTP una y otra vez.

INTRO A WEBSOCKETS



En el backend...

En el frontend...

```
const http = require("http");
const { Server } = require("socket.io");
const Koa = require("koa");

const app = new Koa();
const server = http.createServer(app.callback());

const io = new Server(server, {
  cors: { origin: "*" },
});

io.on("connection", (socket) => {
  console.log("Nuevo cliente conectado:", socket.id);

  socket.on("joinLobby", (gameId) => {
    socket.join(String(gameId));
    console.log(`Jugador unido al lobby ${gameId}`);
  });
});

server.listen(3000, () => console.log("Servidor corriendo 🚀"));
```

```
import { io } from "socket.io-client";
import { createContext, useContext, useEffect } from "react";

const socket = io("http://localhost:3000");
export const SocketContext = createContext(socket);
export const useSocket = () => useContext(SocketContext);

export const SocketProvider = ({ children }) => {
  useEffect(() => {
    return () => socket.disconnect(); // Limpieza al desmontar
  }, []);

  return (
    <SocketContext.Provider value={socket}>
      {children}
    </SocketContext.Provider>
  );
};
```

INTRO A WEBSOCKETS

Comunicación básica

Emitir un evento:

```
socket.emit("joinLobby", String(gameId));
```

Escuchar un evento:

```
socket.on("updateCharacters", (data) => {  
    console.log("Personajes:", data);  
});
```

Dejar de escuchar :

```
socket.off("updateCharacters");
```

INTRO A WEBSOCKETS

Comunicación básica

El servidor puede unir sockets a un mismo **room**:

```
socket.join( "game123" );
```

Y luego emitir un evento a todos los sockets pertenecientes a ese **room**:

```
io.to( "game123" ).emit( "playerJoined",  
                           { name: "Magda" } );
```



¡MUCHAS
GRACIAS!

IIC2513

